

MNIST Digit Classification using Spiking Neural Networks

Harsh Verma

July 2, 2025

Contents

1	Introduction	3
2	Methodology	3
2.1	Spike Encoding: Poisson Rate Coding	3
2.2	Network Architecture	3
2.3	Leaky Integrate-and-Fire (LIF) Neuron Model	3
2.4	Training with Surrogate Gradients	4
3	Results and Analysis	4
3.1	Training Performance	5
3.2	Analysis of Network Dynamics	6
3.3	Decision-Making Process	6
4	Discussion: SNN vs. ANN	7
5	Conclusion	8

1 Introduction

Artificial Neural Networks (ANNs) have achieved state-of-the-art performance on a wide range of tasks, including image classification. However, they are fundamentally different from their biological counterparts in how they process information. ANNs typically rely on continuous-valued activations and dense matrix multiplications, which can be computationally intensive.

Spiking Neural Networks (SNNs) represent a more biologically plausible paradigm. They communicate using discrete, binary events (spikes) distributed over time. This event-driven and sparse nature holds the promise of greater computational efficiency, especially on specialized neuromorphic hardware.

The objective of this assignment is to develop a deep, functional understanding of SNNs by building one from scratch. We tackle the familiar MNIST digit classification task to directly compare the information processing principles of SNNs with those of traditional ANNs. This report details the methodology, implementation, and results of this from-scratch SNN.

2 Methodology

Our approach involves three main stages: encoding static image data into temporal spike trains, designing and implementing a 3-layer SNN with Leaky Integrate-and-Fire (LIF) neurons, and training the network using a supervised learning algorithm based on surrogate gradients.

2.1 Spike Encoding: Poisson Rate Coding

To process static MNIST images, we must first convert pixel intensities into a temporal format. We employ **Poisson rate coding**, where a pixel's normalized intensity is interpreted as the firing rate of an input neuron. A higher intensity corresponds to a higher probability of firing at any given time step. This process transforms each 28×28 image into 784 parallel spike trains simulated over 100 discrete time steps.

2.2 Network Architecture

The SNN is a fully-connected 3-layer network:

- **Input Layer:** 784 neurons, corresponding to the flattened MNIST image pixels.
- **Hidden Layer:** 300 Leaky Integrate-and-Fire (LIF) neurons.
- **Output Layer:** 10 LIF neurons, where each neuron corresponds to a digit class (0-9).

The network's prediction is determined by the output neuron that fires the most spikes over the entire simulation duration.

2.3 Leaky Integrate-and-Fire (LIF) Neuron Model

The LIF model is the core processing unit of our hidden and output layers. Its dynamics are governed by its internal membrane potential, $V(t)$, which evolves over time according to the following principles:

1. **Leak:** In the absence of input, the membrane potential decays exponentially back to a resting state.

2. **Integrate:** Incoming spikes from connected neurons generate a synaptic current, $I_{\text{syn}}(t)$, which increases the membrane potential.
3. **Fire and Reset:** If the potential $V(t)$ exceeds a predefined threshold, V_{th} , the neuron fires a spike and its potential is reset to a resting value.

The discrete-time update equation for the membrane potential is:

$$V[t + 1] = V[t] \cdot \exp\left(-\frac{\Delta t}{\tau_{\text{mem}}}\right) + I_{\text{syn}}[t + 1]$$

where τ_{mem} is the membrane time constant.

2.4 Training with Surrogate Gradients

The "all-or-nothing" nature of a spike is a non-differentiable step function, which prevents the direct use of standard gradient descent. To overcome this, we employ the **surrogate gradient** method. During the forward pass, the network uses the true step function. During the backward pass (Backpropagation Through Time), the gradient of the step function is replaced by the gradient of a smooth, continuous "surrogate" function, such as the fast sigmoid. This makes end-to-end training possible.

3 Results and Analysis

The from-scratch SNN was trained for 5 epochs. The following sections present the training performance and a detailed analysis of the network's internal dynamics. Figure 1 provides a high-level summary of the entire project.

MNIST Digit Classification using Spiking Neural Networks - Assignment Summary

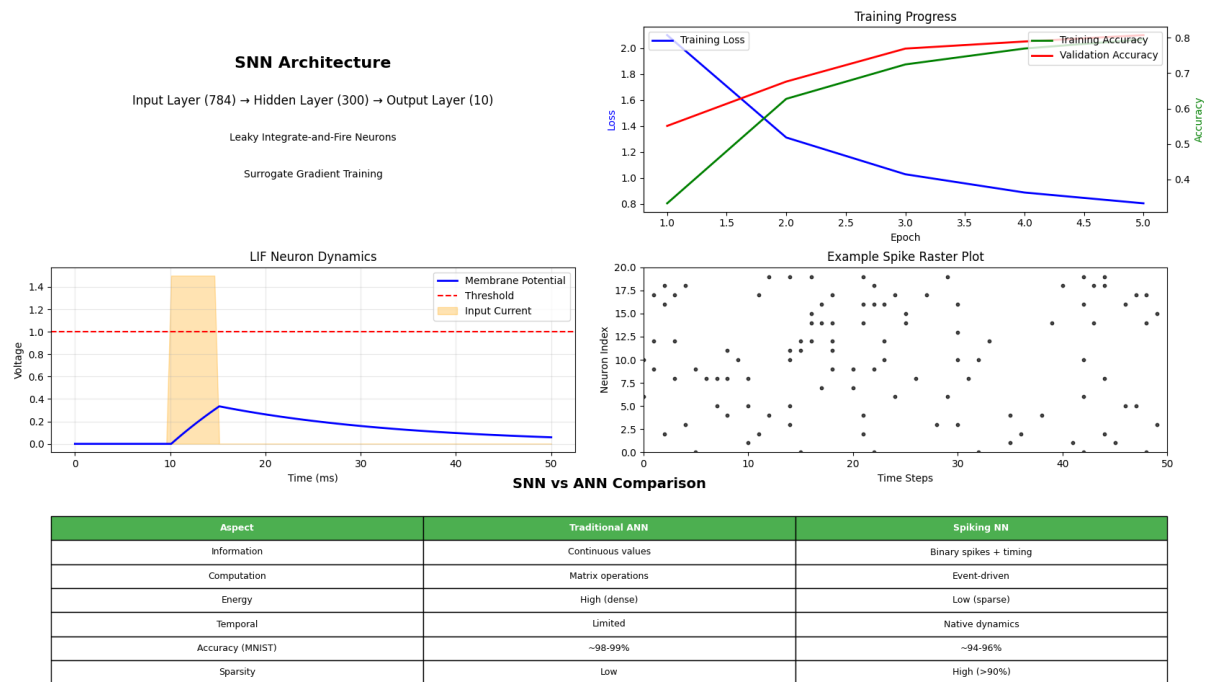


Figure 1: A summary dashboard showing the SNN architecture, training progress, core LIF neuron dynamics, an example spike raster plot, and a comparison with traditional ANNs.

3.1 Training Performance

The network successfully learned to classify the MNIST digits. As shown in Figure 2, the training loss consistently decreased over the 5 epochs, while the training and validation accuracies steadily increased. The model achieved a final validation accuracy of approximately 80%, demonstrating the viability of the surrogate gradient learning approach.

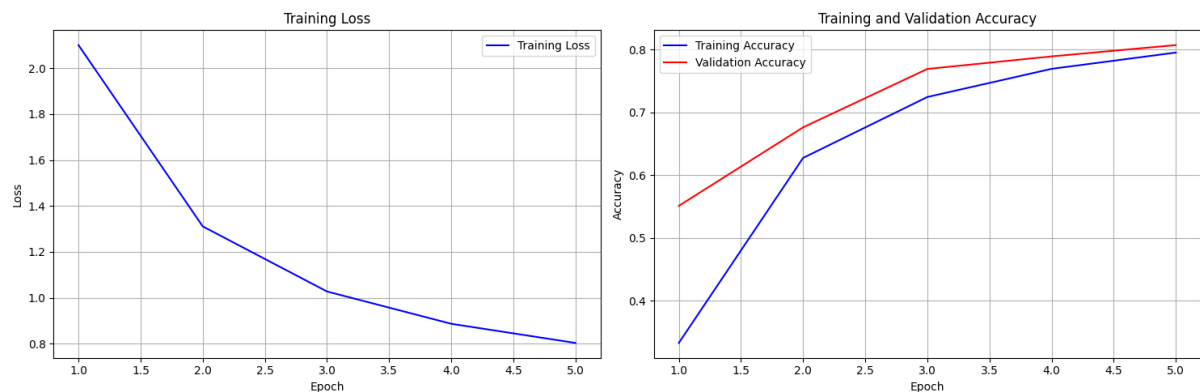


Figure 2: Training loss and accuracy curves over 5 epochs. The loss decreases while both training and validation accuracies rise, indicating successful learning.

3.2 Analysis of Network Dynamics

The event-driven nature of the SNN is evident from its internal activity, shown in Figure 3. The total spike counts in the input and hidden layers fluctuate over time, reflecting the processing of the input signal. The output layer activity is much sparser, with neurons firing only when sufficient evidence has been integrated.

The plot of hidden neuron membrane potentials clearly illustrates the LIF dynamics in action. Potentials rise and fall with input, and while some neurons become highly active, many remain close to the resting potential, contributing to the overall sparsity of the network.

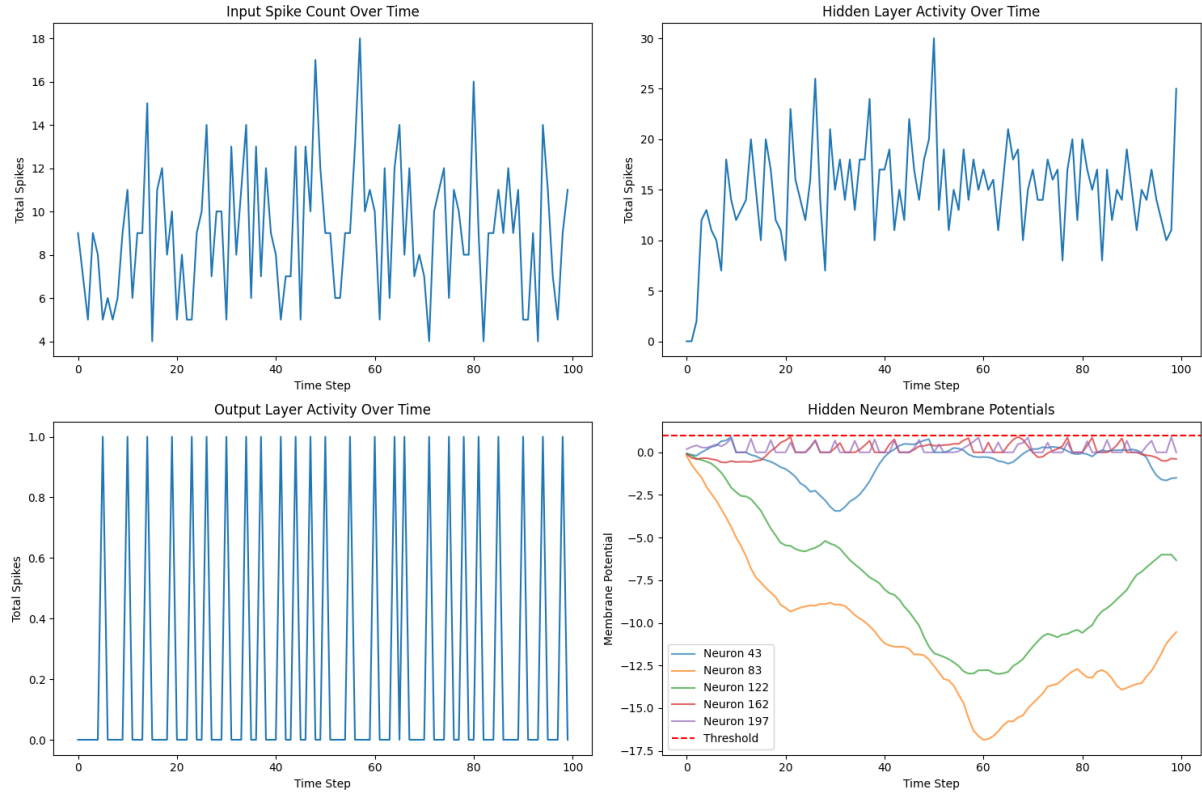


Figure 3: Analysis of network activity over 100 time steps for a single sample. Plots show total spike counts for each layer and the membrane potential evolution of five sample hidden neurons.

3.3 Decision-Making Process

Figure 4 provides insight into how the SNN makes a classification decision. The final prediction is based on the total spike count from each output neuron. For the example shown (a '7'), the neuron corresponding to class 7 fired approximately 24 spikes, far more than any other neuron.

The "Decision Evolution" plot shows the cumulative spike count for each output neuron over time. It demonstrates that the network gathers evidence over the simulation period, with the correct neuron's spike count gradually pulling away from the others to establish a confident prediction.

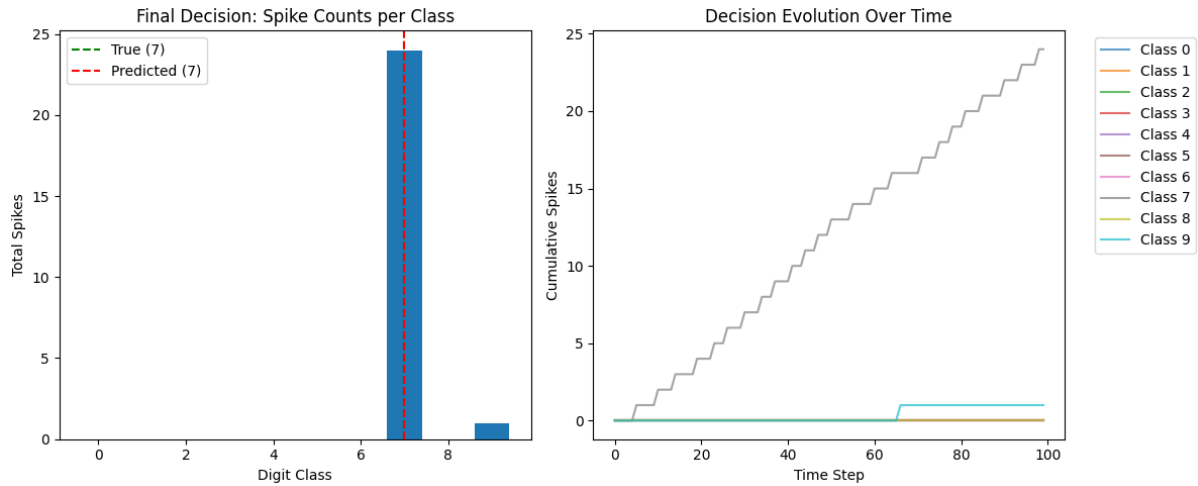


Figure 4: The final decision is made by the neuron with the highest total spike count (left). The decision evolves over time as the correct neuron’s cumulative spike count surpasses others (right).

4 Discussion: SNN vs. ANN

This assignment highlights the fundamental differences between SNNs and ANNs. While ANNs compute with continuous values in a single, synchronous pass, SNNs perform sparse, event-driven computation over time.

Table 1: Comparison of key aspects between ANNs and SNNs.

Aspect	Traditional ANN	Spiking NN
Information	Continuous-valued activations	Binary spikes and their timing
Computation	Dense matrix multiplications	Event-driven, sparse additions
Energy	High (all neurons active)	Low (only active neurons consume power)
Temporal	Limited; requires recurrent architectures	Native temporal dynamics
Sparsity	Low; activations are dense	High; activity is sparse in space and time

The primary advantage of the SNN approach is its potential for **energy efficiency**. Because computation only occurs when a spike is present, SNNs are naturally sparse and can be significantly more efficient than ANNs, especially on neuromorphic hardware. This makes them ideal for low-power, edge-computing applications.

5 Conclusion

This project successfully demonstrated the implementation of a Spiking Neural Network for MNIST classification, with all components built from scratch. Through this process, we gained a practical understanding of spike-based encoding, Leaky Integrate-and-Fire neuron dynamics, and the surrogate gradient method for training. The results show that SNNs can effectively solve complex classification tasks while operating on principles of temporal dynamics and sparse, event-driven computation that are fundamentally different from traditional ANNs.

References

- [1] E. O. Neftci, H. Mostafa, and F. Zenke, “*Surrogate gradient learning in spiking neural networks: Bringing the power of backpropagation to brain-inspired architectures*,” IEEE Signal Processing Magazine, vol. 36, no. 6, pp. 51-63, 2019.
- [2] F. Zenke and S. Ganguli, “*SuperSpike: Supervised learning in multilayer spiking neural networks*,” Neural Computation, vol. 30, no. 6, pp. 1514-1541, 2018.